



UNIVERSIDAD
TORCUATO DI TELLA

Licenciatura en Tecnología Digital

Tecnología Digital VI: Inteligencia Artificial

Aplicación de perceptrones

Clase práctica 15

Motivación

- ¿Cuál es la relación del **aprendizaje profundo** con la inteligencia artificial, el aprendizaje automático, los avances tecnológicos, la biología y la regresión logística?
- ¿Cuál es la diferencia entre un **perceptrón simple**, uno **monocapa** y otro **multicapa**? ¿Cómo afecta la cantidad de neuronas a la **performance** del modelo?
- ¿En qué contextos podemos **aplicar** los perceptrones? ¿Cómo lo hacemos?

Organización

Etapa	Inicio		Duración
	Sección 1	Sección 2	
Introducción	08:00	09:50	05'
Aprendizaje profundo	08:05	09:55	15'
Perceptrón simple	08:20	10:10	25'
Perceptrón monocapa	08:45	10:35	15'
Perceptrón multicapa	09:00	10:50	20'
Cantidad de neuronas	09:20	11:10	05'
Yapa: aplicación en otro contexto	09:20	11:15	05'
Cierre	09:30	11:20	05'
Fin de la clase	09:35	11:25	-

Aprendizaje profundo | Repaso | AI, ML y DL

Recordemos que la **inteligencia artificial** (IA; AI, por sus siglas en inglés), como disciplina, se encarga de comprender y construir entidades inteligentes y que el **aprendizaje automático** (ML, por sus siglas en inglés) refiere a programas de computadoras que, mediante la experiencia (datos), mejoran su rendimiento (valor de métrica de performance) para resolver un conjunto de tareas (objetivo).

Inicialmente, la IA **resolvió de forma rápida** problemas que son intelectualmente difíciles para nosotros, los seres humanos, pero relativamente directos para las computadoras: específicamente, problemas que pueden ser descritos por una lista de **reglas** formales, matemáticas.

El **verdadero desafío** para la IA es, en cambio, resolver tareas que son sencillas de realizar para nosotros pero difíciles para que nosotros las describamos formalmente: concretamente, problemas que nosotros resolvemos **intuitivamente**, que se sienten automáticos, como reconocer palabras que se dicen en una canción o caras que aparecen en una imagen.

La idea es permitirle a las computadoras que aprendan de la experiencia y entiendan el mundo en términos de una **jerarquía de conceptos**, con cada concepto definido mediante sus relaciones con conceptos más simples. Si dibujamos un grafo mostrando cómo estos conceptos se construyen uno sobre el otro, el grafo es profundo, con muchas capas. De ahí el nombre **aprendizaje profundo** (DL, por sus siglas en inglés).

Aprendizaje profundo | Repaso | AI, ML y DL (cont.)

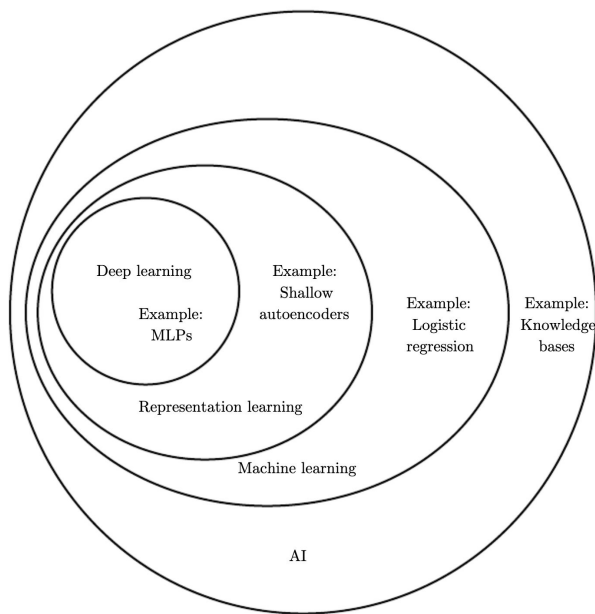


Figure 1.4: A Venn diagram showing how deep learning is a kind of representation learning, which is in turn a kind of machine learning, which is used for many but not all approaches to AI. Each section of the Venn diagram includes an example of an AI technology.

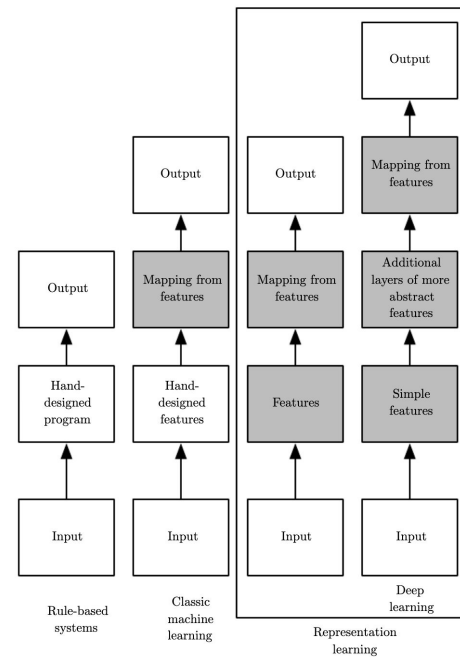


Figure 1.5: Flowcharts showing how the different parts of an AI system relate to each other within different AI disciplines. Shaded boxes indicate components that are able to learn from data.

Aprendizaje profundo | Repaso | Historia

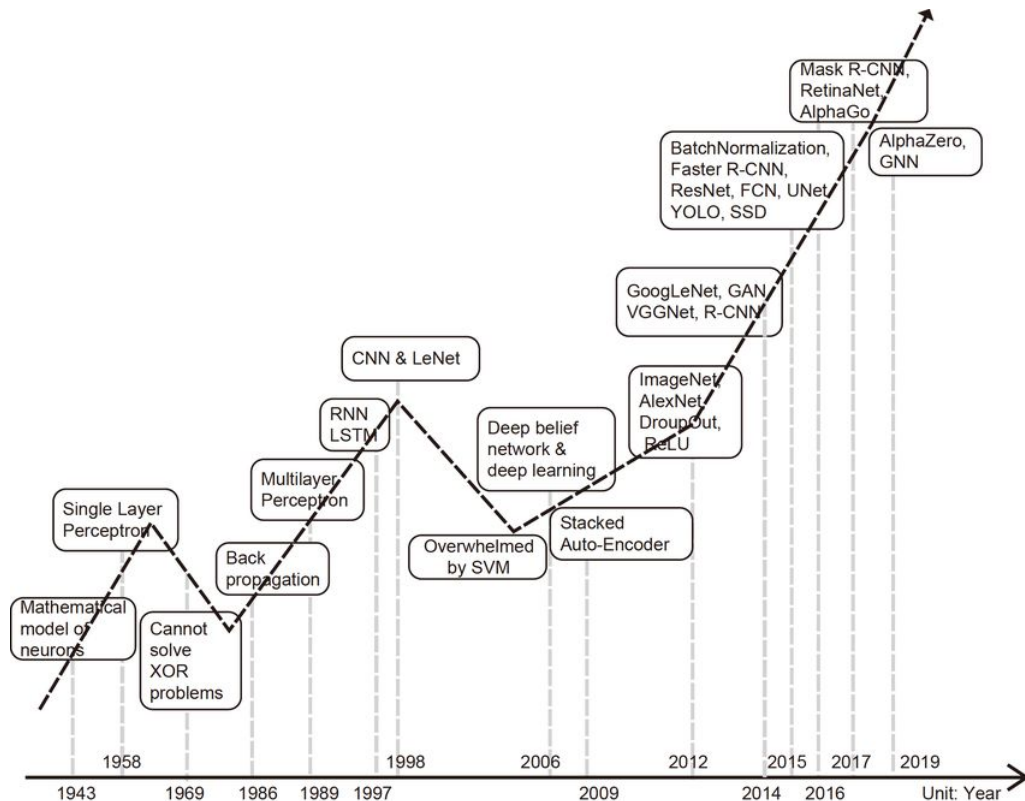
El aprendizaje profundo es un área de **investigación muy activa** en las comunidades de inteligencia artificial y, más específicamente, aprendizaje automático.

La **piedra angular** del aprendizaje profundo es la **red neuronal**.

2000 a 2010. Las redes neuronales se vieron **opacadas** por métodos que surgieron luego que eran más automáticos, requerían menos retoques y superaban la performance de redes pobremente entrenadas, como SVMs, *boosting* y *random forests*.

Después de 2010. Las redes neuronales **resurgieron** con el nuevo nombre *deep learning*, en parte por la disponibilidad de conjuntos de **datos** muchísimo más grandes y por la mayor capacidad de poder de **cómputo**, que facilita a su vez la **experimentación**.

Aprendizaje profundo | Repaso | Historia



Aprendizaje profundo | Repaso | Biología

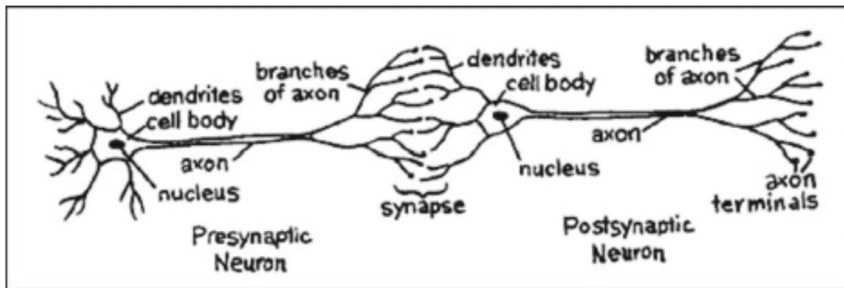
Las **redes neuronales artificiales** son técnicas populares de aprendizaje automático que simulan el mecanismo de aprendizaje en los organismos biológicos.

El sistema nervioso humano contiene células, a las que se refiere como **neuronas**. Las neuronas están conectadas entre ellas mediante **axones** y **dendritas**. Las regiones de conexión entre axones y dendritas se conoce como **sinapsis**. La **fuerza de las conexiones sinápticas** usualmente cambia en respuesta a **estímulos externos**. Este cambio es cómo el **aprendizaje** ocurre en los organismos vivos.

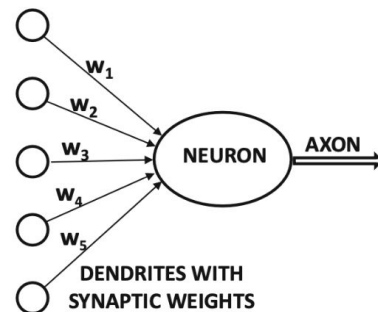
Este mecanismo biológico es simulado en las redes neuronales artificiales:

- Neurona: unidad de **cómputo**.
- Dendritas: por donde viajan los **inputs**.
- Axones: por donde viajan los **outputs**.
- Fuerza de las conexiones sinápticas: **pesos**.
- Estímulo externo: **datos** de entrenamiento.
 - Los datos de entrenamiento proveen **feedback** de cuán correctos son los pesos en la red neuronal dependiendo de cuán bien el output predicho para un input en particular coincide con la etiqueta en los datos de entrenamiento.
 - Los **pesos** entre las neuronas son **ajustados** en la red neuronal en función de los errores de predicción.
 - El **objetivo** de cambiar los pesos es modificar la función calculada para hacer que las **predicciones** sean **más correctas** en futuras iteraciones.
 - Al sucesivamente ajustar los pesos entre las neuronas para muchos pares de input-output, la **función** calculada por la red neuronal será **refinada** en el tiempo para que provea predicciones más precisas.

Aprendizaje profundo | Repaso | Biología (cont.)



(a) Biological neural network



(b) Artificial neural network

Figure 1.1: The synaptic connections between neurons. The image in (a) is from “*The Brain: Understanding Neurobiology Through the Study of Addiction* [598].” Copyright ©2000 by BSCS & Videodiscovery. All rights reserved. Used with permission.

Aprendizaje profundo | Repaso | Regresión

Las **redes neuronales** también pueden ser interpretadas como **abstracciones de los modelos clásicos** de aprendizaje automático, como la regresión logística.

Las redes neuronales adquieren su poder al **combinar muchas de estas unidades básicas** y aprender los pesos de las distintas unidades, de forma conjunta, con el fin de minimizar el error de predicción.

A las redes neuronales se las puede ver como un grafo computacional de unidades elementales en que mayor poder es adquirido al **conectar estas unidades de ciertas formas**.

Importa la **cantidad**: al combinar múltiples **unidades**, uno aumenta el poder del modelo para aprender funciones más complicadas de los datos.

Pero también importa la **calidad**: la forma en que se **combinan** estas unidades juega un rol fundamental en el poder de la arquitectura.

Requisito para cantidad y calidad: contar con **suficientes datos** de entrenamiento.

Aprendizaje profundo | Repaso | Regresión (cont.)

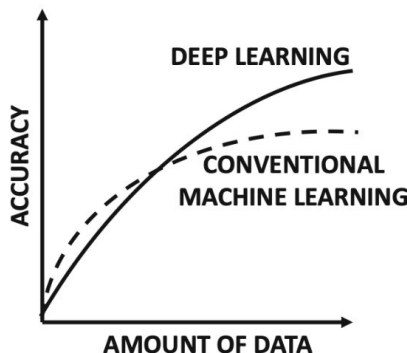
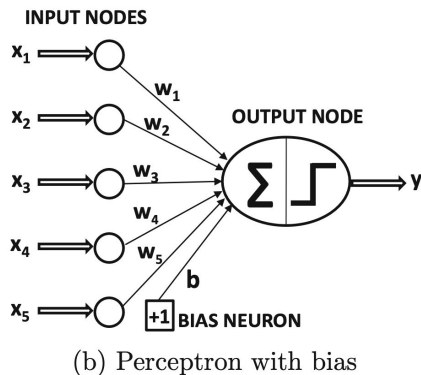
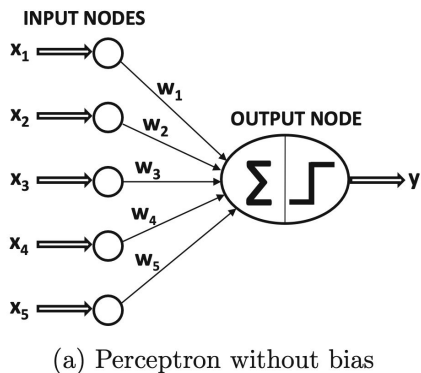


Figure 1.2: An illustrative comparison of the accuracy of a typical machine learning algorithm with that of a large neural network. Deep learners become more attractive than conventional methods primarily when sufficient data/computational power is available. Recent years have seen an increase in data availability and computational power, which has led to a “Cambrian explosion” in deep learning technology.

Perceptrón simple | Repaso

Una **red neuronal** toma un vector de **entrada** de p variables $X = (X_1, X_2, \dots, X_p)$ y construye una **función** no-lineal $f(X)$ para predecir la **respuesta** Y .

Un **perceptrón simple** representa la red neuronal más simple posible: contiene **una sola neurona**. De hecho, no califica como profundo.

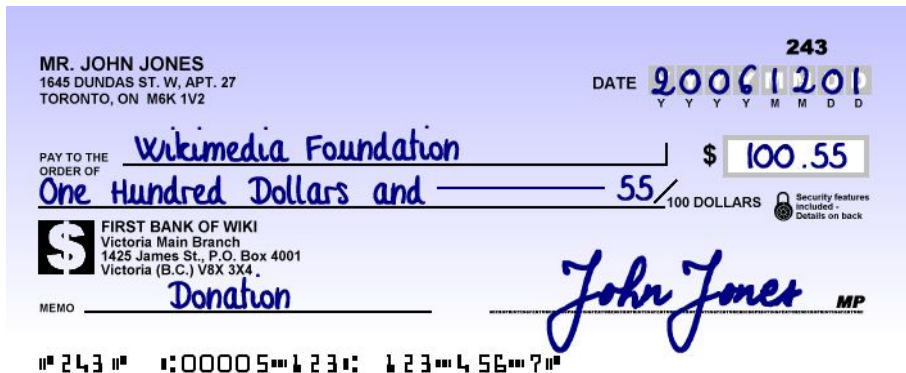


Algunas aclaraciones:

- En la **capa de entrada** en sí misma no se lleva a cabo **ningún cálculo**.
- Después de la **sumatoria**, que junto a los **w** , representa la **combinación lineal**, se aplica la **función de activación**, que en la figura en cuestión se optó por la **función signo**.
- El **sesgo** refiere a la parte invariante de la predicción y es incorporado como un peso a un eje, usando una **neurona de sesgo**, que siempre transmite el valor **1**.

Perceptrón simple | Aplicación

Un banco desea implementar un sistema de **reconocimiento automático de cheques**.

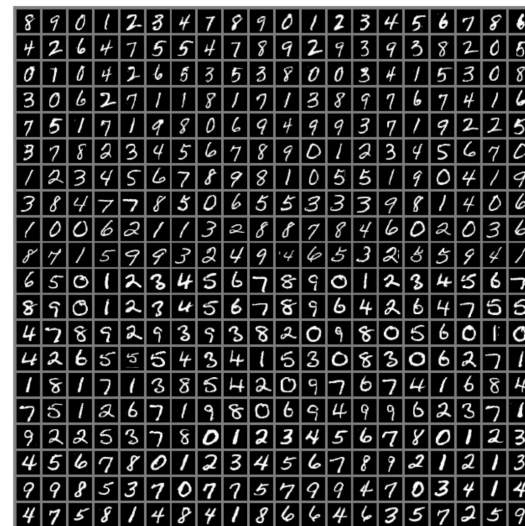


Actualmente, el banco cuenta con un sistema capaz de aislar los dígitos y convertirlos en **imágenes de 28 * 28 píxeles**, pero no de reconocer de qué dígito se trata.

Es en este marco que el banco nos ha contratado para desarrollar un modelo que, dada una imagen de 28 * 28, identifique **de qué dígito manuscrito se trata**.

Perceptrón simple | Aplicación (continuación)

Las imágenes corresponden, concretamente, al popular conjunto de datos conocido como **MNIST** (*modified National Institute of Standards and Technology*).



8	9	0	1	2	3	4	7	8	9	0	1	2	3	4	5	6	7	8	6
4	2	6	4	7	5	5	4	7	8	9	2	9	3	9	3	8	2	0	5
0	1	0	4	2	6	5	3	5	3	8	0	0	3	4	1	5	3	0	8
3	0	6	2	7	1	1	8	1	7	1	3	8	9	7	6	7	4	1	6
7	5	1	7	1	9	8	0	6	9	4	9	9	3	7	1	9	2	2	5
3	7	8	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	0
1	2	3	4	5	6	7	8	9	8	1	0	5	5	1	9	0	4	1	9
3	8	4	7	7	8	5	0	6	5	5	3	3	3	9	8	1	4	0	6
1	0	0	6	2	1	1	3	2	8	8	7	8	4	6	0	2	0	3	6
8	7	1	5	9	9	3	2	4	9	4	6	5	3	2	8	5	9	4	1
6	5	0	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7
8	9	0	1	2	3	4	5	6	7	8	9	6	4	2	6	4	7	5	5
4	7	8	9	2	9	3	9	3	8	2	0	9	8	0	5	6	0	1	0
4	2	6	5	5	5	4	3	4	1	5	3	0	8	3	0	6	2	7	1
1	8	1	7	1	3	8	5	4	2	0	9	7	6	7	4	1	6	8	4
7	5	1	2	6	7	1	9	8	0	6	9	4	9	9	6	2	3	7	1
9	2	2	5	3	7	8	0	1	2	3	4	5	6	7	8	0	1	2	3
4	5	6	7	8	0	1	2	3	4	5	6	7	8	9	2	1	2	1	3
9	9	8	5	3	7	0	7	7	5	7	9	9	4	7	0	3	4	1	4
4	7	5	8	1	4	8	4	1	8	6	6	4	6	3	5	7	2	5	9

Figure 1.9: Example inputs from the MNIST dataset. The “NIST” stands for National Institute of Standards and Technology, the agency that originally collected this data. The “M” stands for “modified,” since the data has been preprocessed for easier use with machine learning algorithms. The MNIST dataset consists of scans of handwritten digits and associated labels describing which digit 0-9 is contained in each image. This simple classification problem is one of the simplest and most widely used tests in deep learning research. It remains popular despite being quite easy for modern techniques to solve. Geoffrey Hinton has described it as “the *drosophila* of machine learning,” meaning that it enables machine learning researchers to study their algorithms in controlled laboratory conditions, much as biologists often study fruit flies.

Perceptrón simple | Aplicación (continuación)

La idea es construir un modelo para **clasificar imágenes** en la clase correcta de **dígito 0-9**.

Hay **60.000** imágenes de **entrenamiento** y **10.000** de **evaluación**.

Cada imagen tiene $p = 28 * 28 =$ **784** **píxeles**.

Cada **pixel** es un valor en una **escala de grises de 8 bits**, entre 0 y 255, representando la cantidad relativa del dígito escrito en ese pequeño cuadrado.

Estos píxeles son almacenados en el **vector de entrada X** .

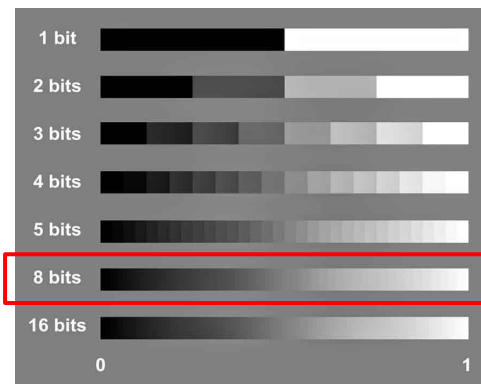


Image Size	Possible combinations (2^k)
1-bit	2
2-bit	4
4-bit	16
8-bit	256
16-bit	65536

8 bits = 256 combinaciones posibles de ceros y unos



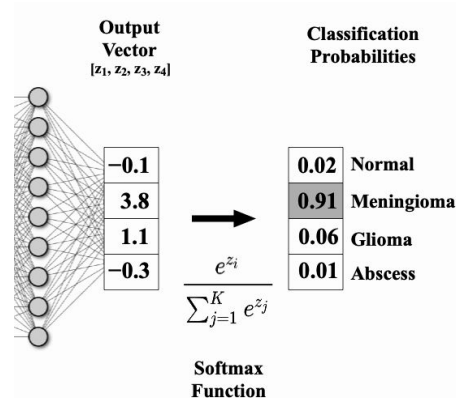
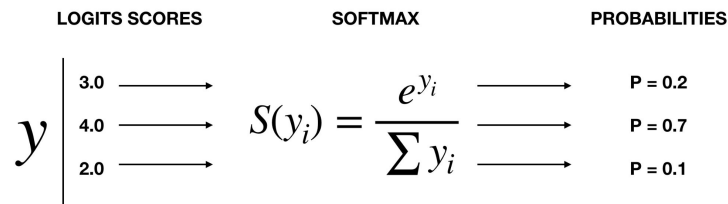
0 0 1 1 0 1 0 0

Perceptrón simple | Aplicación (continuación)

La **salida** es un **vector de 10 variables dicotómicas**, con un 1 en la clase correspondiente y 0s para el resto.

Utilizaremos la **función de activación softmax**, que asegura que las entradas que reciba se transformen en **probabilidades**; es decir, en **valores no-negativos y que suman 1**. (No usamos la función sigmoide porque, si bien genera valores entre 0 y 1, no garantiza que sumen 1).

El modelo estimará, para cada observación, la probabilidad para cada una de las 10 clases y, luego, a cada observación se le asignará **la clase con la mayor probabilidad**. (Si hubiese empate entre dos o más clases posibles, se selecciona una de ellas de forma aleatoria).



Perceptrón simple | Aplicación (continuación)

Veamos `td6-p15-perceptrones.ipynb`.

Perceptrón monocapa | Repaso

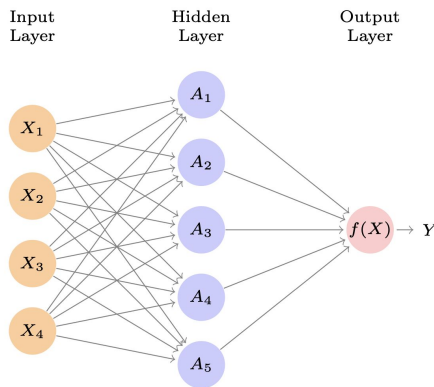


FIGURE 10.1. Neural network with a single hidden layer. The hidden layer computes activations $A_k = h_k(X)$ that are nonlinear transformations of linear combinations of the inputs $X_1, X_2, \dots, X_p$. Hence these A_k are not directly observed. The functions $h_k(\cdot)$ are not fixed in advance, but are learned during the training of the network. The output layer is a linear model that uses these activations A_k as inputs, resulting in a function $f(X)$.

Los p **predictores** conforman las unidades de la **capa de entrada**. Cada una de esas unidades alimenta a las K unidades de la **capa oculta**. K es un hiperparámetro; en la figura, $K = 5$.

Cada A_k es una transformación diferente de los atributos originales. Concretamente, resulta de **combinar linealmente** los atributos originales y, **luego**, a esta combinación aplicarle una función g , conocida como **función de activación**. Es esencial que esta función g sea **no-lineal** para evitar que el modelo colapse en uno lineal simple y para permitir al modelo capturar no-linealidades complejas y efectos de interacción. Los nuevos atributos resultantes son luego alimentados a la(s) unidad(e)s de la **capa de salida**.

Perceptrón monocapa | Repaso (continuación)

Entonces:

$$f(X) = \beta_0 + \sum_{k=1}^K \beta_k \underline{A_k},$$

$$\underline{A_k} = \underline{h_k(X)} = \underline{g(w_{k0} + \sum_{j=1}^p w_{kj} X_j)},$$

Por lo tanto:

$$\begin{aligned} f(X) &= \beta_0 + \sum_{k=1}^K \beta_k \underline{h_k(X)} \\ &= \beta_0 + \sum_{k=1}^K \beta_k \underline{g(w_{k0} + \sum_{j=1}^p w_{kj} X_j)}. \end{aligned}$$

Todos los parámetros β y w deben ser **estimados** a partir de los datos.

Perceptrón monocapa | Aplicación

Volvamos a `td6-p15-perceptrones.ipynb`.

Perceptrón multicapa | Repaso

Las redes neuronales modernas suelen tener **más de una capa oculta** y cada una de ellas con varias neuronas.

Si bien, en teoría, un perceptrón monocapa con un gran número de neuronas en su única capa tiene la habilidad de aproximar casi todas las funciones, el proceso de **aprendizaje** para encontrar una buena solución se torna **mucho más sencillo** si, en cambio, usamos **varias capas, cada una de ellas con una menor cantidad de neuronas**.

La **primera capa oculta** funciona de la misma manera que en el perceptrón monocapa.

Las **siguientes capas ocultas** tratan a las activaciones de la capa oculta anterior como entradas y computan nuevas activaciones.

Perceptrón multicapa | Repaso (continuación)

A la arquitectura específica de un perceptrón multicapa (también conocida como red neuronal multicapa) se la conoce como red **feed-forward** ($\approx$ alimentación hacia adelante), porque capas sucesivas se alimentan entre sí en la dirección de avance desde la entrada hacia la salida.

La arquitectura por defecto de las redes feed-forward asume que todas las neuronas en una capa están conectadas a las neuronas de la capa siguiente. A dichas capas se las conoce como capas **fully-connected** ($\approx$ completamente conectadas).

En la figura de la derecha, podemos ver **distintas formas de representar** a un mismo perceptrón multicapa.

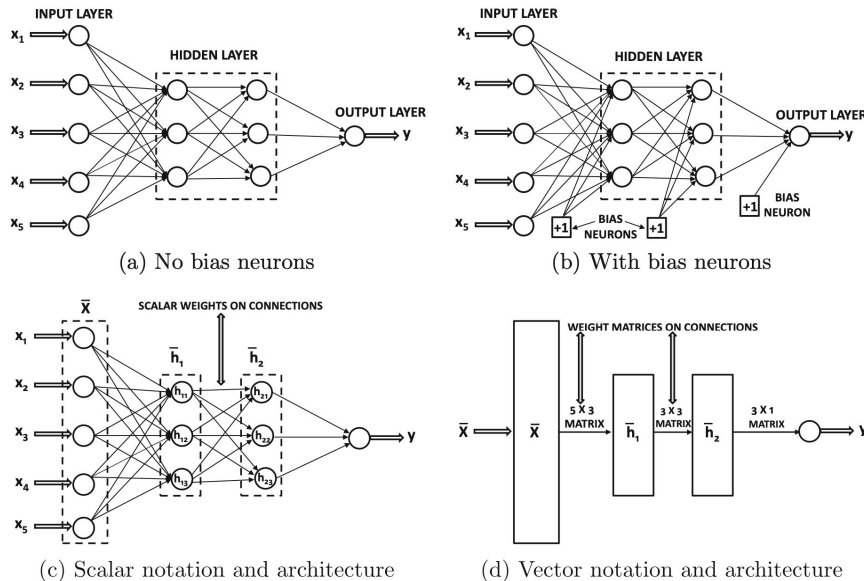


Figure 1.11: The basic architecture of a feed-forward network with two hidden layers and a single output layer. Even though each unit contains a single scalar variable, one often represents all units within a single layer as a single vector unit. Vector units are often represented as rectangles and have connection *matrices* between them.

Perceptrón multicapa | Aplicación

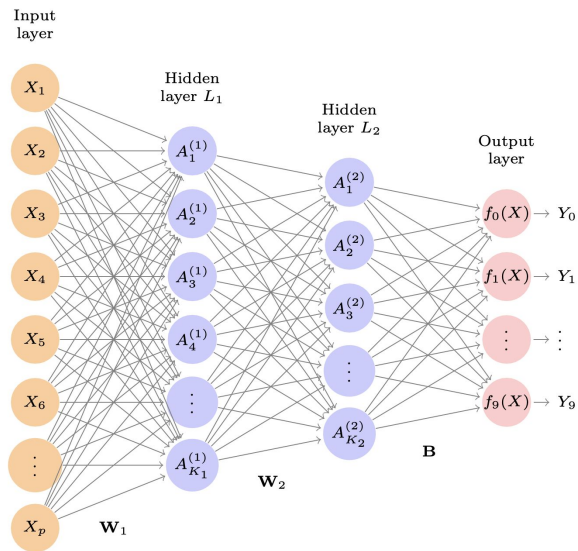


FIGURE 10.4. Neural network diagram with two hidden layers and multiple outputs, suitable for the **MNIST** handwritten-digit problem. The input layer has $p = 784$ units, the two hidden layers $K_1 = 256$ and $K_2 = 128$ units respectively, and the output layer 10 units. Along with intercepts (referred to as biases in the deep-learning community) this network has 235,146 parameters (referred to as weights).

Volvamos a `td6-p15-perceptrones.ipynb`.

La versión completa se encuentra disponible en `td6-p15-perceptrones-completo.ipynb`.

Cantidad de neuronas

Overfitting es un problema importante en el diseño de las **redes neuronales**.

En el ámbito de redes neuronales, ocurre cuando el número de **parámetros libres** es demasiado grande en comparación con la cantidad de **datos de entrenamiento**.

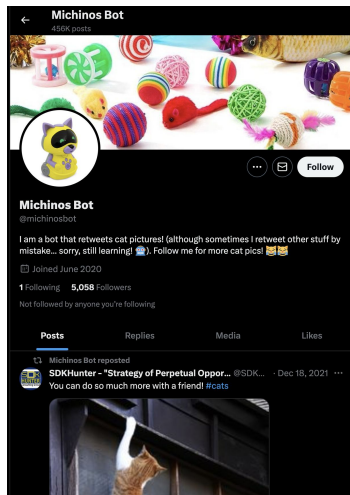
Aumentar el **número de neuronas** tiende a incrementar las chances de cometer overfitting.

Más adelante, vamos a ver varias técnicas, aplicadas a redes neuronales, para lidiar con este problema.

Yapa: aplicación en otro contexto

Si quisiéramos...

... crear un **bot que repostee** publicaciones que muestran una **imagen de un gato** (o de un perro)...



... necesitamos determinar si una imagen corresponde a un gato.

... o desarrollar, para una tienda para mascotas (como [Nutrican](#) o [Puppis](#)), un sistema que, **dada una foto que sube el usuario de su mascota, recomendarle a dicho usuario toda una serie de productos** (e.g., alimento, productos de higiene, cama, juguetes, ropa), la cual varía según si gato/perro (las mascotas más “populares” en Argentina), chico/grande, pelo corto/largo, etc.



... necesitamos determinar si una imagen corresponde o a un gato o a un perro.

Yapa: aplicación en otro contexto (continuación)

Como la cuestión de los bots en $\mathbb{X}$ está complicada, veamos los pasos para determinar si una **imagen corresponde o a un gato o a un perro**, paso fundamental para poder desarrollar el sistema para la tienda de mascotas:

- 1) Obtener **datos etiquetados**; es decir, imágenes, cada una con la etiqueta de si corresponde o a un gato o a un perro. Posibles fuentes: [1](#) y [2](#). Veámoslas.
- 2) Si las imágenes son de tamaños distintos, transformarlas a un **mismo tamaño**.
- 3) Establecer los conjuntos de **entrenamiento, validación y testeo**.
- 4) Entrenar modelos, configurando sus **hiperparámetros** mediante validación y evitando cometer overfitting.
- 5) De los modelos antes entrenados, seleccionar aquel con la **mejor performance** en los datos de validación.
- 6) Estimar la **performance** del modelo ganador en **producción** con los datos de testeo.
- 7) Incorporar el clasificador al **pipeline** del sistema.

Cierre

Hoy vimos y practicamos, entre otras cuestiones,

- cuál es la relación del **aprendizaje profundo** con la inteligencia artificial, el aprendizaje automático, los avances tecnológicos, la biología y la regresión logística;
- cuál es la diferencia entre un **perceptrón simple**, uno **monocapa** y otro **multicapa**; cómo afecta la cantidad de neuronas a la **performance** del modelo y
- en qué contextos podemos **aplicar** los perceptrones y cómo lo hacemos.

La clase que viene vamos a estar practicando **backpropagation**.

Pueden darnos *feedback* de la clase [acá](#).